



Høgskulen
på Vestlandet



DAT108

Programmering og webapplikasjoner

F05 – Mer om streams

Lars-Petter Helland
Bergen
31. august 2026

Plan for dagen

- › Kort oppfrisking av hva vi så på om Streams sist gang
- › Fullføre det vi ikke ble ferdige med (terminal ops: reduce() m.fl.)
- › Om programmeringsparadigmer
- › Optional ←
- › Litt flere operasjoner på Streams
- › Vi avrunder modulen i dag (neste gang starter vi på ny modul: tråder)



Oppfrisking fra sist - Hva er en stream (strøm)

En stream kan tenkes på som et samlebånd av data.

- › Vi kan "sette opp" operasjoner på dataene på samlebåndet, og
- › Vi kan "sette opp" flere operasjoner som utføres etter tur (f.eks. filter() og map()).
- › Til slutt må det være én operasjon som "samlar opp" dataene i et resultat (f.eks. toList()).

Eks:

```
List<String> etternavnene = people.stream() //FROM people
    .filter(p -> p.getAge() > 50)           //WHERE age > 50
    .map(Person::getLastName)                //SELECT lastName
    .toList();
```

Ved å jobbe med dataene på denne måten **unngår vi** både **løkker**, **if-setninger** og **"hvordan"** (add), men benytter oss i stedet av generelle stream-operasjoner og lambda-uttrykk for å spesifisere **"hva"** vi ønsker å gjøre.

```
List<String> etternavnene = new ArrayList<>();
for(Person p : people) {
    if (p.getAge() > 50) {
        etternavnene.add(p.getLastName());
    }
}
```



Oppfrisking fra sist - Source, intermediate og terminal ops

- › Vi har **én kilde(-operasjon)** som forsyner samlebåndet med data:

source

- › Vi har én eller flere operasjoner som utføres på elementene underveis på samlebåndet:

intermediate operations

- › Vi har **én endelig operasjon** på enden av samlebåndet som "samler opp" elementene:

terminal operation



Oppfrisking fra sist - Eksempler på source ops

```
List<Person> personliste = ...;  
Stream<Person> personstream = personliste.stream();  
  
String[] navnetabell = {"Anne", "Per", "Knut"};  
Stream<String> navnestream = Stream.of(navnetabell);  
  
Stream<Integer> tallstream = Stream.of(2,4,5,7,8,9);  
  
IntStream 1til10 = IntStream.range(1,10);  
IntStream 1tilogmed10 = IntStream.rangeClosed(1, 10);  
  
Stream<Integer> uendeligstream = Stream.iterate(1, x -> x + 1);
```



Oppfrisking fra sist - Eksempler på intermediate ops, 1

```
Stream<Person> fornavnPaaCStream  
    = ps.filter(p -> p.getFirstName().startsWith("C"))
```

//**filter** tar et **Predicate** som parameter og lar kun "gyldige" elementer passere

```
Stream<String> etternavnStream = ps.map(Person::getLastName)
```

//**map** tar en **Function** som parameter og transformerer hvert element til noe annet ...

//Dette gir ut en stream med de transformerte elementene. Her **Stream<Person>** ---> **Stream<String>**

```
Stream<...> de10forste = s.limit(10)
```

//**limit** "stenger" streamen når antall elementer er passert. Nødvendig på uendelige streams.



Oppfrisking fra sist - Eksempler på intermediate ops, 2

distinct og sorted, f.eks.:

Eksempel7

```
//d) For en stream av strenger, samle dem opp i en ny string, sortert  
//      og uten duplikater. Skriv deretter ut resultatet.
```

```
System.out.println("\nSorterte strenger uten duplikater");  
String resultat = Stream.of("b", "a", "c", "a", "b")  
    .distinct()  
    .sorted() //evt. sorted(comparator)  
    .collect(Collectors.joining(", "));
```



Oppfrisking fra sist - Eksempler på terminal ops, 1

```
s.forEach(System.out::println);
```

//forEach tar en **Consumer** som parameter utfører denne på alle elementene

```
List<...> nyListe = s.collect(Collectors.toList());
```

//collect: "A mutable reduction operation that accumulates input elements into a mutable result container"

```
List<...> nyListe = s.toList();
```

//toList: En vanlig collect-operasjon som har fått en egen metode. (*toList() = collect(Collectors.toList())*)



reduce(T init, BinaryOperator<T> accu)

Rakk ikke denne sist

- › Vi spanderer et eget lysbilde på reduce.
- › Enda en **terminal operation**. Vi har allerede sett på forEach, toList og collect.
- › På samme måte som **filter** og **map**, er **reduce** en svært sentral metode i funksjonell programmering.
- › Reduce returnerer én verdi basert på elementene i streamen ved å akkumulere opp etter en gitt formel.

```
int sumAlder = people.stream()
    .map(p -> p.getAge())
    .reduce(0, (sum, alder) -> sum + alder);
```

- › Første parameter er startverdi i akkumuleringen
- › Andre parameter er en funksjon som forteller hvordan det skal akkumuleres. Den akkumulerer "i en løkke" til streamen er tom.



Eksempel8 – reduce() m.m.

Rakk ikke denne sist

IntelliJ-prosjekt i dag: 📁 f05-streams2

Pakke for eksempelet: 📦 no.hvl.dat108.f04

Eksempel8

Gjøre/vise



=> Eksempler på terminal ops, 2

reduce ...

sum, average ...

anyMatch, allMatch, noneMatch ...

Se Eksempel8



Programmeringsparadigmer

Fra <https://dataenkelt.com/paradigmer-i-programmering-slik-pavirker-de-maten-du-tenker-kode-pa/>

“Når du lærer å programmere, handler det ikke bare om å lære et språk som Python, Java eller C#. Det handler også om å lære en måte å tenke på – en tilnærming til hvordan problemer skal løses og systemer bygges. Denne tilnærmingen kalles et *programmeringsparadigme*. Paradigmer former hvordan du strukturerer koden din, hvordan du deler opp problemer, og hvordan du vurderer hva som er “god kode”.”

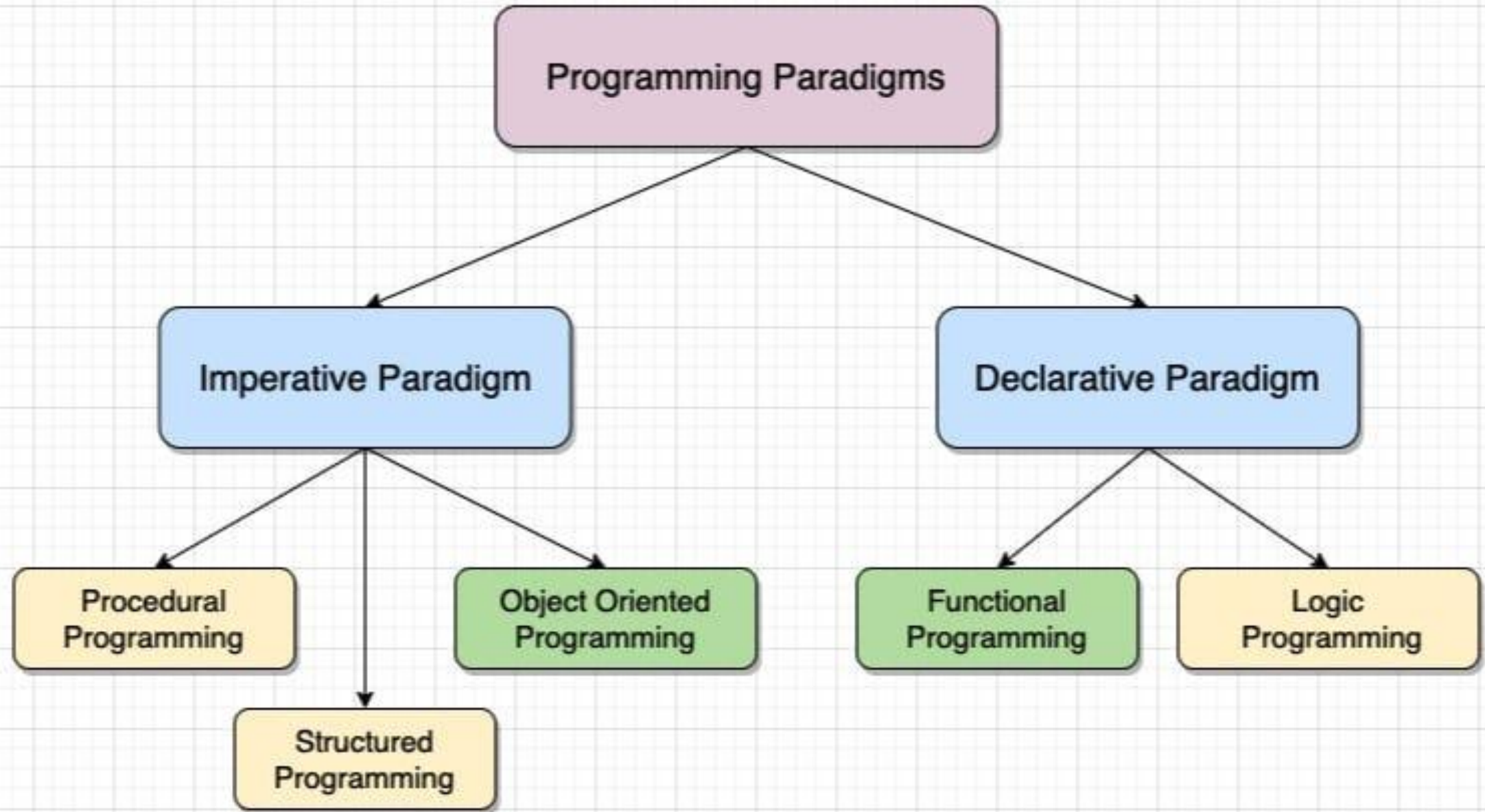
Vanlige paradigmer

Imperative: **Strukturert. Objektorientert. ..**

Deklarative: **Funksjonelt. ..**

Multiparadigme?

De fleste programmeringsspråk har støtte for flere paradigmer. F.eks. lar **Java**, **Python** og **JavaScript** oss programmere både **objektorientert** og **funksjonelt**. Andre språk er mer rendyrkede. F.eks. er **Lisp** og **Haskell** rene funksjonelle språk. Dere kan lære Haskell i et valgfag **INF122** neste høst.



Hva kjennetegner funksjonell programmering?

- › Bruk av **“rene” funksjoner** (pure functions), dvs. funksjoner der svaret kun er avhengig av parametrene. Samme parametre gir samme svar. I tillegg skal funksjonen ikke ha sideeffekter.
- › Bruk av **ikke-muterbare data** (immutable data), dvs. ikke tillate endringer i data etter at de er opprettet i programmet.
- › Bruk av **høyere ordens funksjoner** (higher-order functions), dvs. funksjoner som tar andre funksjoner som parametre og/eller returnerer funksjoner. Funksjoner blir sett på som “data” på samme måte som andre data.
- › Bruk av **rekursjon** (recursion) og **nøstede funksjonskall** (evt. metodekjeding) i stedet for løkker.
- › ...
- › I **Java** vil vi merke variabler med **final**, bruke **record** for å lage klasser, bruke **List.of(..)** når vi oppretter lister, for å få ikke-muterbare data.
- › I **Java** vil vi videre bruke **funksjoner**, **lambda-uttrykk** og **streams** til å lage løsninger og å arbeide med samlinger av data.

Trenger vi egentlig funksjonell programmering?

Noen fordeler med funksjonell vs. imperativ programmering er at vi får:

- › Kode som er mer **fleksibel** (vi kan parametrisere oppførsel)
- › Kode som er mer **lesbar** (høyere abstraksjonsnivå, færre kodelinjer)
- › Kode som er lettere å **vedlikeholde** (færre kodelinjer, bedre modularitet)
- › Kode som er lettere å **teste** (færre variabler, ingen sideeffekter)
- › Kode som er lettere å **parallelisere** (ingen sideeffekter, direkte støtte)

**And...
Let's continue!**



Optional<T>

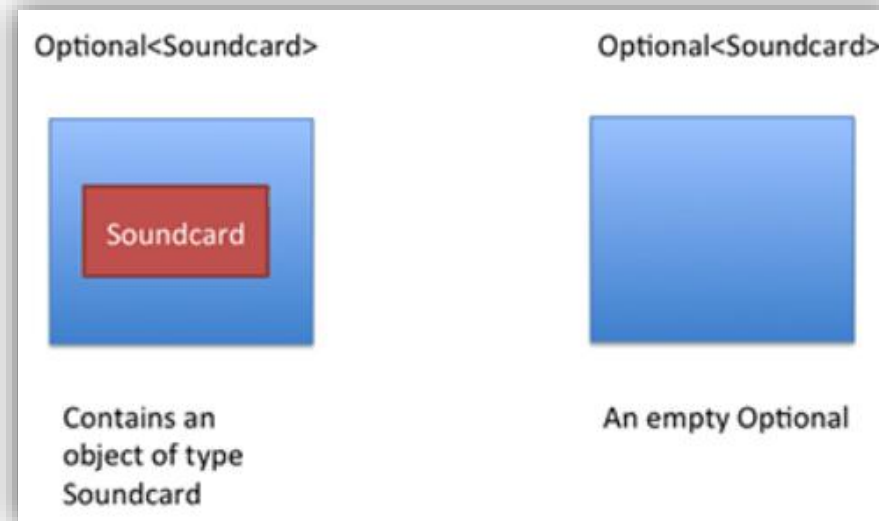
Dere kan lese mer i **Guide To Java 8 Optional**

<https://www.baeldung.com/java-optional> , avsnitt 1-9.

"The purpose of the class is to provide a type-level solution for representing optional values instead of null references."

Eller fra <https://www.oracle.com/technical-resources/articles/java/java8-optional.html>

"You can view Optional as a single-value container that either contains a value or doesn't (it is then said to be "empty")"



Eks1_Optional

IntelliJ-prosjekt i dag: 📁 f05-streams2

Pakke for eksempelet: 📦 no.hvl.dat108.f05

Vise!

- › Stream.min(..)
- › Optional<T>
 - › ifPresentOrElse()
 - › ifPresent()
 - › orElse()
 - › get()
 - › orElseThrow()
- › Viser hvordan Optional kan brukes til å erstatte nullsjekker og unngå NullPointerException.



Eks2_findAny

IntelliJ-prosjekt i dag: 📁 f05-streams2

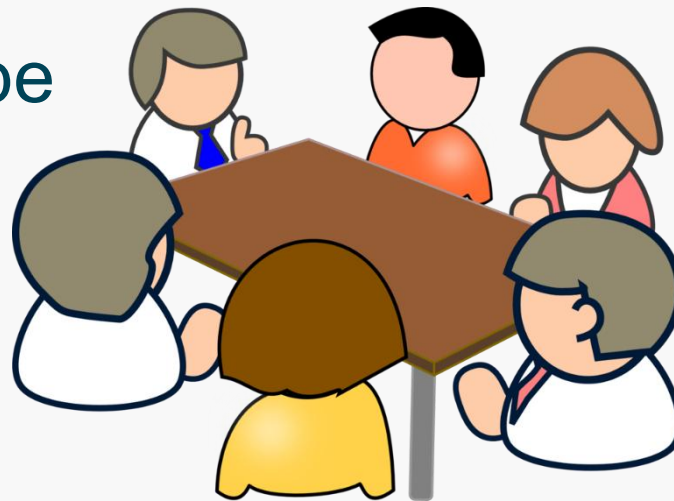
Pakke for eksempelet: 📦 no.hvl.dat108.f05

Gjøre/vise!

- › Optional<T>
- › Stream.findAny(), ..findFirst()
- › Viser at noen terminale operasjoner kan ende opp uten svar. I de tilfeller returneres en Optional i stedet for å returnere mulig null.
- › Vi skal også se senere i kurset at databasespørringer med Spring Data JPA, f.eks. findById(..), returnerer en Optional for å unngå null-retur ved mislykket søk.



Vi trenger en
referansegruppe





Flatmap

Dere kan lese mer i **Java Stream flatMap()**

<https://www.scaler.com/topics/java-flatmap/>

"It is similar to how the map function works. But .. [flatMap] .. also flattens the result into a single collection of items. It is an intermediate operation. Flattening helps in avoiding nested structures.."

Her er kanskje også et par greie eksempler: **Java 8 flatMap example**

<https://mkyong.com/java8/java-8-flatmap-example/>



Eks3_flatMap

IntelliJ-prosjekt i dag: 📁 f05-streams2

Pakke for eksempelet: 📦 no.hvl.dat108.f05

Vise

Basic eksempel



Eks4_utviklere

IntelliJ-prosjekt i dag: 📁 f05-streams2

Pakke for eksempelet: 📦 no.hvl.dat108.f05

Vi skal se på et litt mer komplisert eksempel, og se at vi kan gjøre mer med streams enn det dere allerede har sett.

Vi gjør sammen så godt vi kan 😊



Flatmap og Optional

...

Utgår?



Eks5_Optional_flatMap

Utgår?

IntelliJ-prosjekt i dag: 📁 f05-streams2

Pakke for eksempelet: 📦 no.hvl.dat108.f05

flatMap brukes ikke bare på strømmer ...



Parallele strømmer

...

Utgår?



Eks5_parallel

(hvis vi får tid)

Parallellisering («Bitcoin-mining»)



Det er mye mer, men jeg tror vi sier ...



GOOD ENOUGH!

Videre

Minner om DAT108 Javahjelpen torsdag.

På labben er det fremdeles Oblig1 som gjelder. Frist søndag!

Fredag starter vi med neste modul i kurset: Tråder!

